Android Wi-Fi BT 问题
快速排查指南

版本号: 1.5

发布日期: 2025.09.10

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2019.09.06	AWA0989	文档初始版本
1.1	2021.01.09	AWA0989	转 markdown 格式
1.2	2023.03.21	AWA0989	增加更多实例
1.3	2024.02.22	AWA1828	1. 增加 Wi-Fi 长时间连接异常断连分析指导 2. 增加 iperf UDP 吞吐测试参数注意事项 3. 增加蓝牙因 RTS/CTS 引脚问题导致唤醒异常的案例
1.4	2025.09.09	AWA1828	订正文本错误
1.5	2025.09.10	AWA1828	1. 增加 MAC 地址格式规则要求 2. 增加射频指标强调 3. 增加 WEP 加密配置说明

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**

2 硬件框架 **2**

2.1 芯片内部实现 2

2.2 平台端设计 3

3 软件框架 **5**

3.1 Wi-Fi 子系统 5

3.1.1 Wi-Fi 软件框架 5

3.1.2 技术标准 5

3.2 蓝牙子系统 6

3.2.1 蓝牙软件框架 6

3.2.2 常用 Profile 6

4 常见问题排查指南 **8**

4.1 Wi-Fi 常见问题及调试方法 8

4.1.1 Wi-Fi 打不开 8

4.1.1.1 服务启动异常 8

4.1.1.2 内核驱动异常 8

4.1.1.3 打开自动关闭 11

4.1.2 休眠唤醒类问题 14

4.1.2.1 联网后无法休眠 14

4.1.2.2 网络包无法唤醒系统 15

4.1.2.3 唤醒后断网重连 15

4.1.3 MAC 地址不正确 16

4.1.3.1 MAC 地址定制流程及配置 16

4.1.3.2 MAC 地址定制不生效 17

4.1.4 连接一段时间后断网 18

4.1.5 吞吐率不达标 18

4.1.5.1 共存影响 18

4.1.5.2 调频策略 18

4.1.5.3 硬件指标 19

4.1.5.4 测试环境 19

4.1.6 WEP 加密配置不正确 20

4.2 BT 常见问题及排除方法 20

4.2.1 蓝牙已停止运行 20

4.2.1.1 UART 不通 20

4.2.1.2 LPM 驱动配置 21

4.2.1.3 Firmware 错误 22

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.2 休眠唤醒类问题 22
4.2.2.1 无法休眠 22
4.2.2.2 无法唤醒 23
4.2.2.3 唤醒后挂掉 24
4.2.3 设备兼容性 25
4.2.3.1 LOG 抓取 25
4.2.3.2 所有设备连接困难 26
4.2.3.3 部分设备连接困难 26

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　Wi-Fi/BT 芯片框架 3
图 2-2　Wi-Fi/BT 硬件设计 4
图 4-1　Wi-Fi 关键进程 8
图 4-2　sunxi-rfkill 配置 9
图 4-3　上电时序 9
图 4-4　上电打印 9
图 4-5　手动调试开电 10
图 4-6　SDIO 扫卡 10
图 4-7　SDIO 移除卡 10
图 4-8　手动调试 SDIO 扫卡 11
图 4-9　Firmware 加载失败 11
图 4-10　找不到接口 12
图 4-11　wait_for_driver_ready 接口 12
图 4-12　两个接口共用 wiphy 12
图 4-13　两个接口具有独立 wiphy 13
图 4-14　wlan 接口 flag 13
图 4-15　p2p 接口 flag 13
图 4-16　Realtek 双 wiphy 配置 14
图 4-17　wpa_supplicant 临时修改规避措施 14
图 4-18　休眠期间的广播包 15
图 4-19　DHCP 重启 15
图 4-20　wowlan 配置 16
图 4-21　命令行查看 wowlan 配置 16
图 4-22　MAC DTS 配置 16
图 4-23　命令行查看 mac 地址配置 17
图 4-24　TX/RX 查看 21
图 4-25　LPM 配置 22
图 4-26　异常数据引发蓝牙挂掉 24
图 4-27　TX/RX 上拉电阻 24
图 4-28　HCI log 设置 25
图 4-29　抓空包设置 26

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

- 该文档主要介绍 Android 平台上 Wi-Fi/蓝牙系统的框架，全志对原生框架的修改，及一些常见问题的排查方法。
- 本文档仅用于帮助开发人员快速排查和解决初级问题，如果按照文档中的指导仍无法解决问题，请联系全志原厂 FAE 进一步协助解决，并提供说明是否按照文档中的排查方案进行过必要的排查，提供排查过程的必要记录和数据
- 本文档适用用于 Android O 及以后版本

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 硬件框架

Wi-Fi 蓝牙通常做到一个芯片上。在芯片内部，Wi-Fi、蓝牙通常是相对独立的系统，但为节约成本，其天线通常是共用的。

2.1 芯片内部实现

- **控制系统**

> Wi-Fi：Wi-Fi MCU
> 蓝牙：BT Controller

- **天线分配**

> 2.4/5G：双工器
> TX/RX：FEM/SPXT

- **通信接口**

> SDIO/UART：AP6xxx、RTL8723CS、XR829、AW869A、AW869B
> USB：RTL8723DU、AW869AU
> SDIO：AW859(UWE5622)
> PCIE：RTL8188CE

版权所有 © 珠海全志科技股份有限公司。保留一切权利

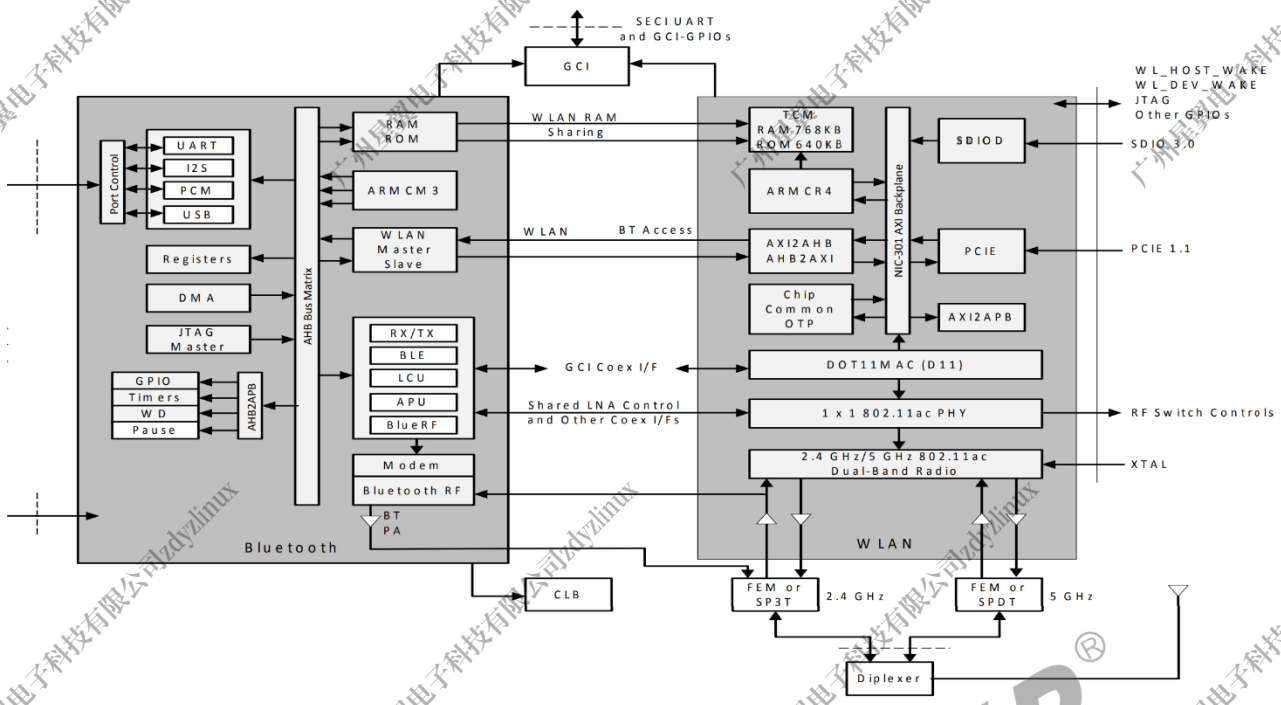

图 2-1: Wi-Fi/BT 芯片框架

2.2 平台端设计

以目前广泛应用的 SDIO+UART 模组为例，在软件对接时，主要关注的硬件信号如下：

表 2-1: Wi-Fi/BT 主要硬件信号

类别	名称	作用
供电	VCC-WIFI	芯片供电
	VCC-IO-WIFI	IO 供电
控制引脚	WL_REG_ON	Wi-Fi 使能
	BT_RSTN	蓝牙 RESET
休眠唤醒	WL_WAKE_AP	Wi-Fi wakeup AP
	BT_WAKE	AP wakeup BT
	BT_HOST_WAKE	BT wakeup AP
通信接口	SDIO_D0/D1/D2/D3/CMD/CLK	SDIO
	UART_RX/TX/RST/CTS	4 线 UART
	PCM_DIN/DOUT/CLK/SYNC	PCM 接口
时钟信号	XTAL_IN/OUT	高速时钟
	LPO	32K 时钟

版权所有 © 珠海全志科技股份有限公司。保留一切权利

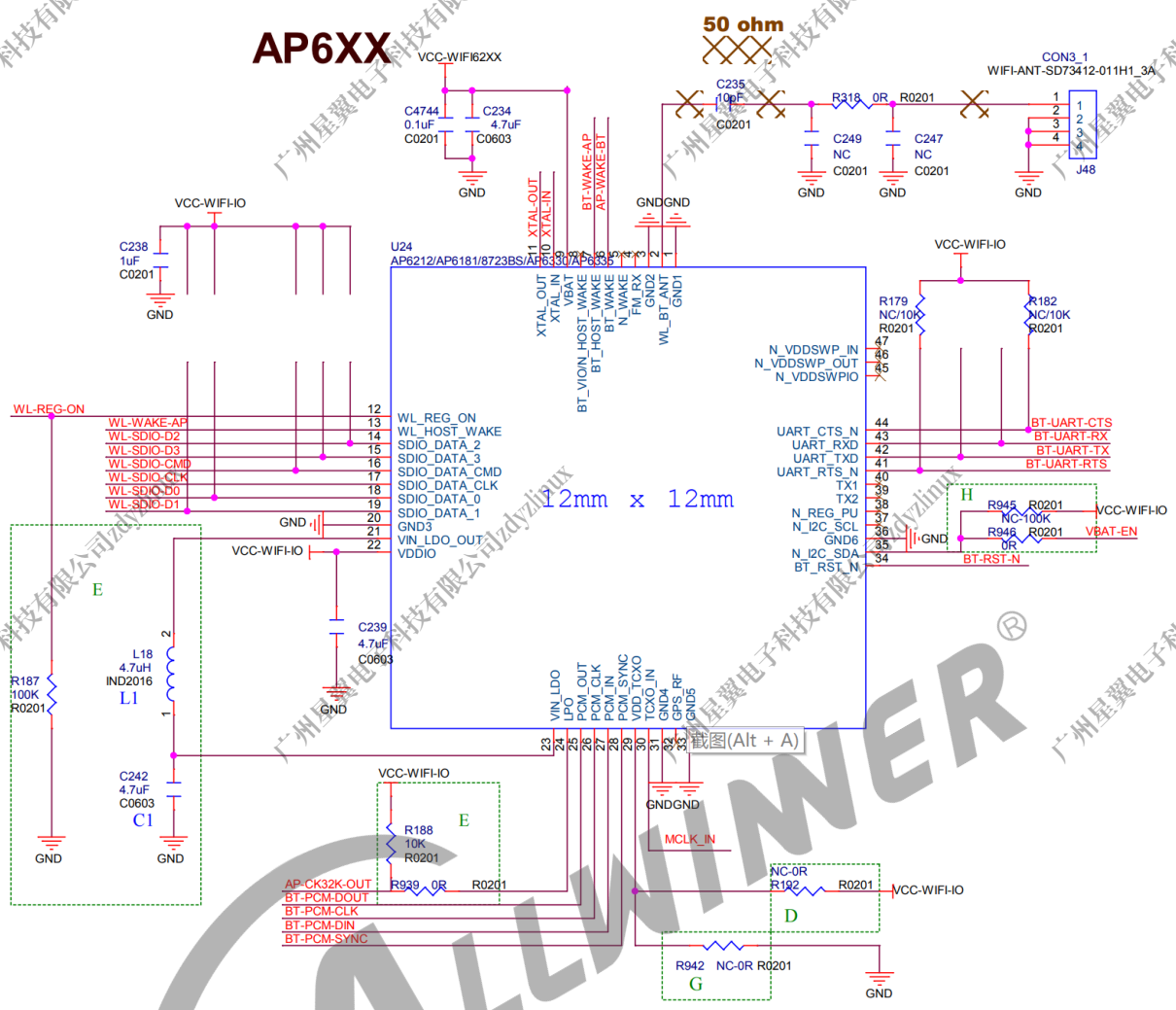

图 2-2: Wi-Fi/BT 硬件设计

广州星翼电子科技有限公司zdylinux

3 软件框架

3.1 Wi-Fi 子系统

3.1.1 Wi-Fi 软件框架

- **Wi-Fi HAL**

> common：Google 框架，通常厂商无需修改，通过编译链接静态库的方式与厂商对接。
> Vendor: 厂商实现，静态库。

- **wpa_supplicant**

> 作用：认证/连接管理，扫描已经逐渐移至 wificond
> Priv-lib：私有命令扩展，静态库

- **Driver**

> FullMAC：对接 cfg80211_ops
> SoftMAC：对接 ieee80211_ops

3.1.2 技术标准

表 3-1: Wi-Fi 技术标准

协议	时间	频段	信道宽度/MHz	MIMO	基础速率/Mbsp	最大速率/Mbps
802.11b	1999	2.4G	20	1x1	11	11
802.11a	1999	5G	20	1x1	54	54
802.11g	2003	2.4G	20	1x1	54	54

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

协议	时间	频段	信道宽度/MHz	MIMO	基础速率/Mbsp	最大速率/Mbps
802.11n	2009	2.4/5G	20/40	4x4	150 (HT40@1x1)	600
802.11ac	2011	5G	20/40/80/160	8x8	433.3 (HT80@1x1)	6928
802.11ax	2019	2.4/5G	20/40/80/160	8x8	600.5 (HT80@1x1)	9608

3.2 蓝牙子系统

3.2.1 蓝牙软件框架

- **Profile/Stack**

> A2DP/HID/OPP/HFP：JAVA 层具体功能实现，Google 实现
> Bluedroid：Google 维护

- **Libbt**

> 与 stack/profile 无关
> 直接与 controller 交互

- **Btlpm**

> 低功耗控制：使蓝牙尽快进入休眠；蓝牙唤醒 AP

- **Firmware**

> RoM Patch：Realtek/Broadcom 等，仅针对芯片 ROM 打补丁
> Full Firmware：XR829/XR819S，完整的 Firmware

3.2.2 常用 Profile

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 3-2: 蓝牙常用 Profile

类别	名称	解释
蓝牙音乐	A2DP-Sink/AVRCP-CT	蓝牙音乐接收端，音箱等
	A2DP-Source/AVRCP-TG	蓝牙音乐发送端，平板等
外设	HID	非 BLE 类型的 HID 设备，鼠标键盘等
	HOGP	HID 在 BLE 上的实现
蓝牙通话	HFP-AG	语音通话手机、平板端实现
	HFP-HS	语音通话耳机端实现
文件传输	OPP	文件传输
网络共享	PAN	网络共享

广州星翼电子科技有限公司zdyzlinux

4 常见问题排查指南

4.1 Wi-Fi 常见问题及调试方法

4.1.1 Wi-Fi 打不开

4.1.1.1 服务启动异常

Android 上 Wi-Fi 打开后必须存在如下两个进程：android.hardware.wifi@1.0-service 及 wpa_supplicant。可通过点击 Wi-Fi 按钮，查看 logcat 的方式确认是否有起这两个进程。如无相关进程，可根据具体报错确定引发进程启动失败的原因。如下面 log，原因是加载驱动失败：

```
1844  1844 E android.hardware.wifi@1.0-service: finit_module return: -1: Permission denied
1844  1844 E android.hardware.wifi@1.0-service: Failed to load WiFi driver
1844  1844 E android.hardware.wifi@1.0-service: Failed to initialize firmware mode controller
1844  1844 E android.hardware.wifi@1.0-service: Wifi HAL start failed
3472  4675 E HalDevMgr: Cannot start IWifi: 9 ()
3472  5386 E HalDevMgr: IWifiEventCallback.onFailure: 9 ()
3472  4675 E WifiVendorHal: Failed to start vendor HAL
3472  4675 E WifiNative: Failed to start vendor HAL
3472  4675 E WifiNative: Failed to start Hal
```

图 4-1: Wi-Fi 关键进程

4.1.1.2 内核驱动异常

1. **排查模组上电是否正常**

Wi-Fi 的时钟、供电主要由 sunxi-rf 驱动控制。支持的配置，及典型的上电时序如下所示：

广州星翼电子科技有限公司zdyzlinux

```
&rfkill {
    compatible = "allwinner,sunxi-rfkill";
    chip_en;
    power_en;
    pinctrl-0;
    pinctrl-names;
    status = "okay";

    /* wlan session */
    wlan {
        compatible     = "allwinner,sunxi-wlan";
        clocks         = <&rtc_ccu CLK_DCXO24M_OUT>, <&rtc_ccu CLK_OSC32K_OUT>;
        clock-names    = "dcxo24M-out", "osc32k-out";
        wlan_power     = "axp803-dldo1";
        wlan_power_vol= <3300000>;
        wlan_busnum    = <0x1>;
        wlan_regon     = <&r_pio PL 5 GPIO_ACTIVE_HIGH>;
        wlan_hostwake = <&r_pio PL 6 GPIO_ACTIVE_HIGH>;
        wakeup-source;
    };

    /* bt session */
    bt {
        compatible     = "allwinner,sunxi-bt";
        clocks         = <&rtc_ccu CLK_DCXO24M_OUT>, <&rtc_ccu CLK_OSC32K_OUT>;
        clock-names    = "dcxo24M-out", "osc32k-out";
        bt_power       = "axp803-dldo1";
        bt_power_vol   = <3300000>;
        bt_rst_n       = <&r_pio PL 2 GPIO_ACTIVE_LOW>;
    };
};
```

图 4-2: sunxi-rfkill 配置

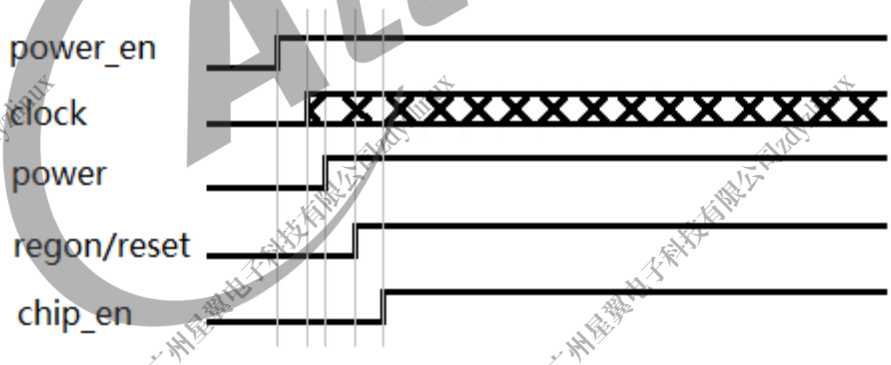

图 4-3: 上电时序

上电成功后，应该有 sunxi-rfkill 的相关打印，典型如下：

```
[16307.957082] sunxi-rfkill soc@2900000:rfkill: wlan power on success
```

图 4-4: 上电打印

如果 Wi-Fi 驱动加载失败，电会自动关闭。此时，可以通过写节点的方式开启 Wi-Fi 供电，以便硬件排查：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
ceres-b6:/ # echo 1 > /sys/devices/virtual/misc/sunxi-wlan/rf-ctrl/power_state
[16513.098496] sunxi-rfkill soc@2900000:rfkill: wlan power on success
```

图 4-5: 手动调试开电

手动开启 Wi-Fi 供电后，可以万用表确认各路电是否正常。

2. 排查 SDIO 扫卡是否正常

驱动加载模组上电后的首要动作是执行一次扫卡动作，以确认卡片（Wi-Fi 芯片）是否存在。当模组上电状态正确时，原则上，SDIO 应该能够扫到设备，内核应有如下打印：

```
[16307.957082] sunxi-rfkill soc@2900000:rfkill: wlan power on success
[16308.013206] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm UP vdd 23 width 1 timing LEGACY(SDR12) dt B
[16308.013367] sunxi-mmc 4021000.sdmmc: no vqmmc,Check if there is regulator
[16308.026094] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16308.039805] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[16308.048311] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[16308.055977] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16308.059051] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16308.060793] sunxi-mmc 4021000.sdmmc: exceeding card's volts
[16308.074472] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 11, RE !!
[16308.082036] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm OFF vdd 0 width 1 timing LEGACY(SDR12) dt B
[16308.086398] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm UP vdd 21 width 1 timing LEGACY(SDR12) dt B
[16308.098881] sunxi-mmc 4021000.sdmmc: no vqmmc,Check if there is regulator
[16308.110883] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 21 width 1 timing LEGACY(SDR12) dt B
[16308.149280] mmc2: queuing unknown CIS tuple 0x10 (5 bytes)
[16308.155291] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 50000000Hz bm PP pm ON vdd 21 width 1 timing LEGACY(SDR12) dt B
[16308.155479] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 50000000Hz bm PP pm ON vdd 21 width 4 timing LEGACY(SDR12) dt B
[16308.155677] mmc2: queuing unknown CIS tuple 0x10 (5 bytes)
[16308.157186] mmc2: new SDIO card at address 626a
```

图 4-6: SDIO 扫卡

当驱动卸载时，模组断电，也会执行一次扫卡动作，以确认卡片（Wi-Fi 芯片）已移除，内核打印如下：

```
[16617.919535] sunxi-rfkill soc@2900000:rfkill: wlan power off success
[16618.019623] aicbsp: aicbsp_platform_power_off
[16618.019632] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 7, RTO !!
[16618.027182] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 7, RTO !!
[16618.034770] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 7, RTO !!
[16618.042311] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 7, RTO !!
[16618.050238] mmc2: card 08d3 removed
[16618.050351] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm OFF vdd 0 width 1 timing LEGACY(SDR12) dt B
[16618.053787] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm UP vdd 23 width 1 timing LEGACY(SDR12) dt B
[16618.062790] sunxi-mmc 4021000.sdmmc: no vqmmc,Check if there is regulator
[16618.074707] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16618.087613] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[16618.096080] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[16618.103748] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16618.109917] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16618.112135] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[16618.120646] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[16618.129103] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[16618.137587] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[16618.145183] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm OFF vdd 0 width 1 timing LEGACY(SDR12) dt B
```

图 4-7: SDIO 移除卡

同样，也可以手动触发一次扫卡动作，以确认扫卡是否正常：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
ceres-b6:/ # echo 1 > /sys/devices/virtual/misc/sunxi-wlan/rf-ctrl/power_state
ceres-b6:/ # [16830.298745] sunxi-rfkill soc@2900000:rfkill: wlan power on success
ceres-b6:/ # echo 1 > /sys/devices/virtual/misc/sunxi-wlan/rf-ctrl/scan_device
ceres-b6:/ # [16837.125255] misc sunxi-wlan: start scan device on bus_index: 1
[16837.127594] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm UP vdd 23 width 1 timing LEGACY(SDR12) dt B
[16837.127868] sunxi-mmc 4021000.sdmmc: no vqmmc,Check if there is regulator
[16837.138859] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16837.151518] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[16837.160122] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[16837.168072] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16837.174135] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 23 width 1 timing LEGACY(SDR12) dt B
[16837.176095] sunxi-mmc 4021000.sdmmc: exceeding card's volts
[16837.211551] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 11, RE !!
[16837.219300] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm OFF vdd 0 width 1 timing LEGACY(SDR12) dt B
[16837.222091] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm UP vdd 21 width 1 timing LEGACY(SDR12) dt B
[16837.222305] sunxi-mmc 4021000.sdmmc: no vqmmc,Check if there is regulator
[16837.234762] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 21 width 1 timing LEGACY(SDR12) dt B
[16837.264426] mmc2: queuing unknown CIS tuple 0x10 (5 bytes)
[16837.269812] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 50000000Hz bm PP pm ON vdd 21 width 1 timing LEGACY(SDR12) dt B
[16837.269958] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 50000000Hz bm PP pm ON vdd 21 width 4 timing LEGACY(SDR12) dt B
[16837.270172] mmc2: queuing unknown CIS tuple 0x10 (5 bytes)
[16837.274845] mmc2: new SDIO card at address e325
```

图 4-8: 手动调试 SDIO 扫卡

3. **Firmware 加载**

驱动一般都会显式的打印出 Firmware 加载过程，如有 Firmware 加载失败的问题，可从打印中看出。如下 log，表面 Firmware 加载失败，失败原因是 Firmware 文件未找到：

```
[17142.831878] rwnx_request_firmware, name: fw_adid.bin
[17142.832512] rwnx_request_firmware, name: fw_patch.bin
[17142.850623] rwnx_request_firmware, name: fw_patch_table.bin
[17142.868430] aicbt_patch_table_load bt uart baud: 1500000, flowctrl: 1, lpm_enable: 1, tx_pwr: 24608
[17142.881027] aicbsp: bt patch version: - Jul 19 2022 19:29:50 - git abd6b22
[17142.881293] (NULL device *): Direct firmware load for fmacfw.bin failed with error -2
[17142.881303] (NULL device *): Falling back to sysfs fallback for: fmacfw.bin
[17142.887696] ueventd: firmware: loading 'fmacfw.bin' for '/devices/virtual/firmware/fmacfw.bin'
[17142.903842] ueventd: firmware: could not find firmware for fmacfw.bin
```

图 4-9: Firmware 加载失败

注意，Realtek 模组 Firmware 是内嵌在驱动里面的，无需获取额外的 Firmware 文件支持。

4. **接口注册失败**

如驱动加载无明显报错，但驱动加载后 ifconfig -a 并未发现 wlan0 接口存在，则一般是驱动与平台兼容性问题，请联系全志获取帮助。

注意，有的 Wi-Fi 在驱动加载后，仅注册接口，但接口是 down 的状态，ifconfig 看不到 wlan0，ifconfig -a 才可以看到，如博通。

4.1.1.3 打开自动关闭

1. **驱动接口异步注册**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

典型的如 Realtek、ssv 的模组，在驱动加载返回时，并不意味着接口已经注册完成。此时，Wi-Fi HAL 启动时找不到 wlan0 接口，造成驱动加载后又被马上卸载掉的问题。

```
07-25 08:30:21.159   460   660 I WifiVendorHal: Vendor Hal started successfully
07-25 08:30:21.165   700   700 W android.hardware.wifi@1.0-service-lazy: No active wlan interfaces in use! Using default
07-25 08:30:21.167   460   660 E HalDevMgr: Error on getCapabilities_1_5: 9 (unknown error)
07-25 08:30:21.170   700   700 E android.hardware.wifi@1.0-service-lazy: Failed to add interface: wlan0 NOT_SUPPORTED
07-25 08:30:21.171   460   660 E HalDevMgr: executeChipReconfiguration: failed to create interface createIfaceType=0: 4 ()
07-25 08:30:21.172   460   660 D HalDevMgr: createIfaceIfPossible: Failed to create iface for ifaceType=0, requestorWs=WorkSource{1000 com.android.settings}
07-25 08:30:21.172   460   660 E WifiVendorHal: Failed to create STA iface
07-25 08:30:21.172   460   660 E WifiNative: Failed to create iface in vendor HAL
```

图 4-10: 找不到接口

此时，需要 HAL 驱动加载后，等待 wlan0 接口稳定，再继续跑 HAL 的相关流程。AndroidP 及之后，google 框架实现了 wait_for_driver_ready，即是这个作用。AndroidP 之前版本，可参考该代码进行修改。

```
||      wifi_cleanup                 || 151 /*initialize function pointer table with Realtek HAL API*/
||      internal_pollin_handler      || 152 wifi_error init_wifi_vendor_hal_func_table(wifi_hal_fn *fn)
||      wifi_event_loop              || 153 {
||      internal_no_seq_check        || 154     if (fn == NULL) {
||      internal_valid_message#      || 155         return WIFI_ERROR_UNKNOWN;
||      GetMulticastIdCommand #      || 156     }
||      getId [GetMulticastIdC#      || 157     fn->wifi_initialize = wifi_initialize;
||      create [GetMulticastId#      || 158     fn->wifi_wait_for_driver_ready = wifi_wait_for_driver_ready;
||      handleResponse [GetMul#      || 159     fn->wifi_cleanup = wifi_cleanup;
||      SetPnoMacAddrOuiComman#      || 160     fn->wifi_event_loop = wifi_event_loop;
||      createRequest [SetPnoM#      || 161     fn->wifi_get_supported_feature_set = wifi_get_supported_feature_set;
||      start [SetPnoMacAddrOu#      || 162     fn->wifi_get_concurrency_matrix = wifi_get_concurrency_matrix;
||      handleResponse [SetPno#      || 163     fn->wifi_set_scanning_mac_oui =  wifi_set_scanning_mac_oui;
||      SetNodfsCommand [SetNo#      || 164     fn->wifi_get_ifaces = wifi_get_ifaces;
__Tag_List__                          hardware/realtek/wlan/wifi_hal/rtw_wifi_hal.cpp
```

图 4-11: wait_for_driver_ready 接口

2. **驱动不满足规范**

AndroidP 及之后，wpa_supplicant 增加对 wlan、p2p 接口 flag 的检查。在注册 wlan0 接口时，如果发现 wlan0 接口所在 wiphy 具有 **NL80211_IFTYPE_P2P_CLIENT** 及 **NL80211_IFTYPE_P2P_GO** 标志位，则会导致 wpa_supplicant 中认为该接口是一个 p2p 接口，服务启动失败，造成驱动加载后又被马上卸载掉的问题。

此时，需要查看驱动加载后接口注册情况。通常，一个 phy 中注册两个接口 wlan、p2p 的情况是不满足 Android 规范的，因为此时两个接口的 flag 是一致的。当两个接口分别注册在两个 phy 下时，需要看具体的接口 flag。

```
ceres-b6:/ # iw dev
phy#4
        Interface p2p0
                ifindex 20
                wdev 0x400000002
                addr 44:f5:ef:c3:5e:0a
                type managed
        Interface wlan0
                ifindex 19
                wdev 0x400000001
                addr 44:f5:ef:c3:5e:0b
                type managed
```

图 4-12: 两个接口共用 wiphy

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
ceres-b10:/ # iw dev
phy#1
        Interface p2p0
                ifindex 16
                wdev 0x100000001
                addr 02:e0:4c:33:ab:74
                type managed
phy#0
        Interface wlan0
                ifindex 15
                wdev 0x1
                addr 66:12:e0:7d:51:f9
                type managed
```

图 4-13: 两个接口具有独立 wiphy

一般情况下，两个接口共用 wiphy 不符合当前的框架设计，驱动应避免这种情况。当两个接口具有独立 wiphy 时，Android 要求的典型接口 flag 如下：

```
Wiphy phy0
        max # scan SSIDs: 9
        max scan IEs length: 2304 bytes
        max # sched scan SSIDs: 0
        max # match sets: 0
        Retry short limit: 7
        Retry long limit: 4
        Coverage class: 0 (up to 0m)
        Supported Ciphers:
                * WEP40 (00-0f-ac:1)
                * WEP104 (00-0f-ac:5)
                * TKIP (00-0f-ac:2)
                * CCMP (00-0f-ac:4)
                * CMAC (00-0f-ac:6)
        Available Antennas: TX 0 RX 0
        Supported interface modes:
                 * IBSS
                 * managed
                 * AP
```

图 4-14: wlan 接口 flag

```
ceres-b10:/ # iw phy1 info
Wiphy phy1
        max # scan SSIDs: 9
        max scan IEs length: 2304 bytes
        max # sched scan SSIDs: 0
        max # match sets: 0
        Retry short limit: 7
        Retry long limit: 4
        Coverage class: 0 (up to 0m)
        Supported Ciphers:
                * WEP40 (00-0f-ac:1)
                * WEP104 (00-0f-ac:5)
                * TKIP (00-0f-ac:2)
                * CCMP (00-0f-ac:4)
                * CMAC (00-0f-ac:6)
        Available Antennas: TX 0 RX 0
        Supported interface modes:
                 * IBSS
                 * managed
                 * AP
                 * P2P-client
                 * P2P-GO
```

图 4-15: p2p 接口 flag

目前 Realtek 新版本驱动默认配置为单个接口下注册了 wlan0 及 p2p0，存在该问题，可以通过修改内核配置：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
diff --git a/drivers/net/wireless/rtl8723cs/include/drv_conf.h b/drivers/net/wireless/rtl8723cs/include/drv_conf.h
index f3c90f525c8e..e9bd6222b45c 100644
--- a/drivers/net/wireless/rtl8723cs/include/drv_conf.h
+++ b/drivers/net/wireless/rtl8723cs/include/drv_conf.h
@@ -73,7 +73,9 @@
 #endif

 /* Default enable single wiphy if driver ver >= 5.9 */
+/*
 #define RTW_SINGLE_WIPHY
+ */

 #ifdef CONFIG_RTW_ANDROID
```

图 4-16: Realtek 双 wiphy 配置

对于 SSV 等驱动不肯做变动的厂商，wpa_supplicant 的规避修改（AndroidQ 及之后）：

```
diff --git a/wpa_supplicant/wpa_supplicant.c b/wpa_supplicant/wpa_supplicant.c
index ff3a9646..2f61418c 100644
--- a/wpa_supplicant/wpa_supplicant.c
+++ b/wpa_supplicant/wpa_supplicant.c
@@ -6528,6 +6528,9 @@ static int wpa_supplicant_init_iface(struct wpa_supplicant *wpa_s,
        if (wpa_s->max_remain_on_chan == 0)
                wpa_s->max_remain_on_chan = 1000;

+       if (strncmp(wpa_s->ifname, "wlan0", 4) == 0)
+               wpa_s->drv_flags |= WPA_DRIVER_FLAGS_DEDICATED_P2P_DEVICE
+
        /*
         * Only take p2p_mgmt parameters when P2P Device is supported.
         * Doing it here as it determines whether l2_packet_init() will be done
```

图 4-17: wpa_supplicant 临时修改规避措施

4.1.2 休眠唤醒类问题

4.1.2.1 联网后无法休眠

1. **关于网络唤醒的认知**

Wi-Fi 只要接收到数据包，且该数据包由 Wi-Fi 自己无法处理时，就会通过唤醒引脚唤醒系统，通知上层处理。这类数据包到来时的行为如同按了 power 按键，是不可屏蔽的。

2. **排查唤醒行为是否合理**

灭屏状态下串口抓取通信数据包，一段时间后导出使用 wireshark 分析

```
tcpdump -i wlan0 -w /sdcard/dump.pcap &
```

如，休眠期间的广播包实际可以不交由上层处理。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

23	39.246200	::	ff02::16	ICMPv6	110	Multicast Listener Report Message v2
24	39.631391	::	ff02::16	ICMPv6	110	Multicast Listener Report Message v2
25	40.036912	HuaweiDe_c6:de:00	Spanning-tree-(for-brid…	0x0000	44	Ethernet II,
26	40.314409	fe80::3b60:f693:98a1:182d	ff02::2	ICMPv6	70	Router Solicitation from b4:38:e9:3c:b8:c9
27	40.314627	fe80::3b60:f693:98a1:182d	ff02::16	ICMPv6	90	Multicast Listener Report Message v2
28	40.449016	192.168.11.13	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.251 for any sources

图 4-18: 休眠期间的广播包

4.1.2.2 网络包无法唤醒系统

1. **power/io_regulator 休眠不能断电**

平台支持 Wi-Fi 中断唤醒的前提是，Wi-Fi 在机器休眠期间不能断电，与之连接的 SDC、GPIO 也不应该断电。首先应该排查机器进入深度休眠后各路供电是否正常。可通过直接测量对应引脚电压的方式进行确认。

2. **host_wake 具有唤醒能力**

目前全志平台支持中断唤醒的 IO 引脚为 PL、PM，请务必确保硬件上模组中断输出引脚连接到了这两组 GPIO 上。注意，wlan_hostwake 配置为外部中断唤醒引脚，接模组的中断输出引脚。

3. **休眠期间发生了断网**

休眠期间发生断网行为。可以观察亮屏一瞬间，是否有 Wi-Fi 重连的现象，或者看 log 是否有 DHCP 分配地址的行为：

```
07-25 09:21:52.596   760   922 D DhcpClient: DHCP Packet Handler stopped
07-25 09:21:52.600   760   922 D DhcpClient: onQuitting
07-25 09:21:56.896   760   922 D DhcpClient: l2key: Wa54843f608a52652 lease address: /192.168.11.38 lease expiry: 1658798357718 current time: 1658712116896
07-25 09:21:56.900   760   922 D DhcpClient: Broadcasting DHCPREQUEST ciaddr=0.0.0.0 request=192.168.11.38 serverid=null
07-25 09:21:56.939   760   922 D DhcpClient: Received packet: 44:f5:ef:c3:5e:0b ACK, your new IP /192.168.11.38, netmask /255.255.255.0, gateways [/192.168.11.1] DNS servers: /192.168.11.1 ,
07-25 09:21:56.942   760   922 D DhcpClient: Confirmed lease: android.net.networkstack.DhcpResults@2f1d6f4 DHCP server /192.168.11.1 Vendor info null lease 86400 seconds Servername
07-25 09:21:56.991   760   922 D DhcpClient: Scheduling renewal in 43199s
07-25 09:21:56.991   760   922 D DhcpClient: Scheduling rebind in 75599s
```

图 4-19: DHCP 重启

4.1.2.3 唤醒后断网重连

1. **power/io_regulator 休眠不能断电**

同上。

2. **wpa_supplicant 配置**

Android 上配置 Wi-Fi 是否能够唤醒系统的文件为 wpa_supplicant_overlay.conf。其原理是，通过 wpa_supplicant 下发命令到 kernel 空间，请求 wowlan 为使能状态。wowlan 使能状态下，机器进入休眠后 Wi-Fi 可保持连接，当接收到数据后，会通过中断引脚唤醒系统。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

若关闭 wowlan，机器在进入休眠后会主动断开网络连接，从而无法通过网络唤醒机器。

Wowlan 典型的配置内容如下：

```
ceres-b6:/ # cat /vendor/etc/wifi/wpa_supplicant_overlay.conf
disable_scan_offload=1
wowlan_triggers=any
p2p_disabled=0
filter_rssi=-75
no_ctrl_interface=
```

图 4-20: wowlan 配置

可通过 iw 命令查看 wowlan 状态是否使能：

```
ceres-b6:/ # iw phy0 wowlan show
WoWLAN is enabled:
 * wake up on special any trigger
```

图 4-21: 命令行查看 wowlan 配置

4.1.3 MAC 地址不正确

4.1.3.1 MAC 地址定制流程及配置

支持 MAC 地址定制的平台，MAC 地址定制流程如下

其 MAC 地址来源由 DTS 配置决定。支持的配置及含义如下：

```
&addr_mgt {
    compatible      = "allwinner,sunxi-addr_mgt";
    type_addr_wifi  = <0x0>;
    type_addr_bt    = <0x0>;
    type_addr_eth   = <0x0>;
    status          = "okay";
};
```

0: type_any
1: type_burn
2: type_idgen
3: type_user
4: type_rand

图 4-22: MAC DTS 配置

在系统上，可以通过命令行确定当前传递到 sunxi-addr 驱动的地址

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@dragonboard:~# cat /sys/class/addr_mgt/summary
name cfg cur address
wifi  0   2  E4:29:48:96:BE:75
  bt  0   2  01:DB:60:1F:DB:BC
 eth  0   2  A0:0C:32:5D:B6:9B
```

图 4-23: 命令行查看 mac 地址配置

4.1.3.2 MAC 地址定制不生效

如果发现当前系统 Wi-Fi/BT 的 MAC 地址与烧写的不一致，一般有如下原因：

1. **烧写地址格式错误，未生效**

MAC 地址由六个字节组成，互相之间用冒号 “:” 隔开。如 10:11:12:13:14:15 是合规的，而 10,11,12,13,14,15 则是违规的。

MAC 地址的首个字节中，最低有效位（第 8 位）表示地址类型：
0：单播地址 (unicast)
1：多播地址 (multicast)
次低有效位（第 7 位）表示地址范围：
0：全局管理地址 (global)
1：本地管理地址 (local)

因为很多 Wi-Fi P2P 接口的 MAC 地址使用 WLAN0 地址生成，差异在于一个是 global address，一个是 local address。为了避免造成两个接口地址一致，全志 MAC 地址定制限制了只能烧写 global address，即：第一个 byte 的低两位都必须为 0。如果烧写的地址不满足规范，sunxi-addr 驱动校验不过不会使用。如 10:11:12:13:14:15 是一个合法地址，11:11:12:13:14:15、12:11:12:13:14:15 则是一个非法地址。

2. **所使用的模组不支持定制 MAC 地址**

Broadcom 驱动 MAC 地址需要 Firmware 的校验，需要跟模组厂沟通，因此一般不提供定制。其他厂商的新移植的驱动，请确认驱动中是否调用了 get_custom_mac_address 接口，如无，则不支持。

3. **开启了随机 MAC 地址功能**

AndroidQ 及之后为保障隐私，Google 实现了 MAC 地址随机化功能，保障连接到不同热点上的 MAC 地址不同。平板默认开启。如果看到的 MAC 地址是个 local address，则一般是一个随机地址。通过设置内的开关可以关闭该功能。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.1.4 连接一段时间后断网

1. **DHCP 租赁到期**

对于使用 DHCP 分配 IP 方式的连接中，当 DHCP 分配的 IP 租赁期到期后，STA 与 AP 的连接会先断开，再重新去获取一次 IP。如果获取成功，Wi-Fi 是保持连接的；但如果获取失败则 Wi-Fi 会断连。可通过判断 Wi-Fi 断开前是否有 DHCP 相关打印，来判断此次断连是否为 DHCP 异常造成。

2. **GTK 更新过于频繁**

在 Wi-Fi 信号良好 (rssi <= 50) 情况下，容易出现连接断开，且断开的 reason code 为 7、16 等与鉴权认证失败相关的原因时，可在系统日志中判断 GTK 打印是否频繁出现。一般来说，其每次更新频率会在两个小时或以上。如果出现几分钟、十几分钟更新一次的，则可认为更新过于频繁。GTK 用于双方鉴权认证，如果某一次 GTK 更新在 STA 端未接收到，则会被 AP 踢出。频繁的更新 GTK 会放大 STA 丢掉漏掉更新包的概率，进而导致断连。

4.1.5 吞吐率不达标

4.1.5.1 共存影响

1. **Wi-Fi/蓝牙共存**

由于 Wi-Fi、蓝牙通常共用天线，而同一时刻只能有一方能使用天线，蓝牙的优先级往往又比 Wi-Fi 高，因此，开启了蓝牙且蓝牙有活跃的情况下，吞吐率一般会有比较明显的下降。可关闭确认。

2. **多业务共存**

该情况特指单开 Wi-Fi 的情况。如果测试时 UI 处于 Settings 界面，则每隔 10s，系统会发起一次 scan 动作，scan 优先级相对来说更高，会打断 Wi-Fi 应用数据的传输，扫描期间吞吐率会有大幅度的下降。

如果安装了 GMS 服务，且开启了位置服务，此时后台可能存在比较频繁的扫描动作，请确认关闭。

4.1.5.2 调频策略

1. **CPU 动态调频**

AW 调频策略机制，调频时，先把 CPU 频率拉低最低值，再逐渐往上调到目标频率。这个过程上层是不可见的。因此，只要发生调频动作，此时吞吐量就会有较大的波动，造成平均数据下降。

解决方法：AW869 实现了向用户空间发送 uevent，申请 CPU 定频一段时间。检测到需要较大的网络带宽时便会触发该机制。其他未实现的驱动可以固定 CPU 频率测试对比排查。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. **DDR 调频**

A133 Android12 开启 DDR 调频后，黑屏状态下 DDR 频率被拉到 99M，此时 Wi-Fi 吞吐率奇差。可以亮屏进行测试。或者关闭 DDR 调频机制。

4.1.5.3 硬件指标

射频是 Wi-Fi/BT 的重要硬件指标，它会影响到吞吐性能表现、关键步骤（如认证、连接、回连）的成功率等。我们**强烈建议**每个新的板型都要进行射频指标的检查。

1. **频偏**

晶体频率精度是 Wi-Fi 正常工作的保证。频率精度过低，会导致各个信道中心频点频偏，数据帧无法正确解析，误包率高，出现无法关联或者吞吐量低的问题。进行频偏校准时，可选取 1，6，12 信道做校准即可。如果频偏较大，可通过调整晶体匹配电容做频率调整。

2. **传导**

传导测试体现的是射频信号的板级性能，如 Wi-Fi 外围电路设计是否合理、∏ 型网络参数是否正确等。如果测试达标，可直接通过传导线往 Wi-Fi 芯片注入数据，看吞吐率是否达标。

3. **辐射**

辐射测试体现的是整机性能，包含天线设计是否合理。如果传导测试达标，可重点排查天线部分，或交由天线厂做天线匹配。

4.1.5.4 测试环境

1. **屏蔽房**

建议在屏蔽房进行吞吐率测试，以排除其他射频信号的干扰。如无条件，可选择干扰较少的非办公环境。

2. **测试 AP 性能**

测试 AP 的性能不得低于 Wi-Fi 模组本身的性能，否则会造成瓶颈，无法得到正确的结果。另外，需要注意测试 AP LAN 口速率。如果测试 AP6255 等 5G AC 模组，测试 AP LAN 口速率不得低于 1G，否则也会造成瓶颈。其他影响因素，诸如是否支持 Short GI、是否支持 HT40/HT80/HT160 等，也需要根据具体模组进行确认。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3. **测试主机性能**

注意测试主机的 LAN 口速率与测试 AP LAN 口匹配，均要高于 Wi-Fi 模组速率。另外，测试过程中需要关闭需要网络连接的软件、服务，以避免造成影响。

4. **UDP 参数**

对于 UDP 测试，如果输入 -b 带宽参数过高，会导致出现吞吐数据掉零的情况。具体参数范围可参考 TCP 测试结果来进行小幅度变动。

另外，对于部分测试环境，单一线程的吞吐量可能会受限。例如 xx 办公笔记本 + xx 版本 iperf3，单一线程吞吐上限为 70M。那么在测试更高的带宽时，需要开启多线程。例如测试 120M 的吞吐时，输入参数应从 `-b 120M` 改为 `-b 60M -P 2`。

4.1.6 WEP 加密配置不正确

WEP 密钥由初始化向量以及有效密钥两个部分组成，其中初始化向量固定占有 24 位，剩余有效密钥则是我们要填写的部分。例如我们在配置密钥类型为 64bit 时，64bit - 24bit = 40bit = 10 个十六进制密码长度 = 5 个 ASCII 密码长度。128 位、152 位配置计算方式类似，可以直接参考下面表格：

密钥类型 \密钥格式	十六进制	ASCII
64 位	10	5
128 位	26	13
152 位	32	16

4.2 BT 常见问题及排除方法

4.2.1 蓝牙已停止运行

蓝牙已停止运行的根本原因是，UART 链路不通，命令超时。Android Bluetooth Stack 中会检查 HCI 命令是否超时。如果超时，则会引发代码调用 abort() 接口，将该进程停止掉，进而引发蓝牙服务挂掉。Google 对此解释是，不要尝试恢复通信，如果是软件或硬件原因引起的命令超时，请修复它。

4.2.1.1 UART 不通

1. **检查 UART 端口/波特率**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

对于 Broadcom/XRADIO 平台，使用 bt_vendor.conf/vnd_<device>.txt 配置文件进行端口及波特率配置。其中，vnd_<device>.txt 为编译配置文件，bt_vendor.conf 为运行配置文件。运行配置文件可以在 libbt 运行时覆盖掉编译配置里面的参数。

对于 Realtek，使用 rtkbt.conf 配置文件配置 UART 端口。注意，Realtek 的波特率需要在 rtl8xxxxx_config 的文件中配置。该文件为二进制格式，如需更改波特率，请联系全志或 Realtek 提供支持。

2. **检查 UART 权限**

UART 接口权限会在 initrc 中设置。典型的，UART 应设置为如下权限及用户、组：

```
crw-rw---- 1 bluetooth net\_bt\_admin 247, 1 2019-09-08 13:00 /dev/ttyS1
```

3. **RTS/CTS 使用**

目前全志平台发布的 SDK 默认不支持两线 UART，请在硬件设计时避免该情况。如有特殊需求（IO 不够用等），请联系全志获取额外的驱动及 Firmware 支持。

如以上检查没有问题，可通过 shell 命令获取 UART 当前的配置，及数据收发状态。正常情况下，蓝牙打开时，对应 UART 的 tx、rx 数据均应该是增长的：

```
console:/proc/tty/driver # stty -F /dev/ttyS1
speed 1500000 baud; line = 0;
-parenb -parodd -cmspar -cstopb crtscts
-brkint -icrnl -imaxbel
-opost
-isig -icanon -iexten -echo
console:/proc/tty/driver # cat uart
serinfo:1.0 driver revision:
0: uart:SUNXI mmio:0x05000000 irq:198 tx:179707 rx:1201 fe:1 RTS|DTR
1: uart:SUNXI mmio:0x05000400 irq:199 tx:316753 rx:1613 pe:1 RTS|CTS|DTR
```

图 4-24: TX/RX 查看

4.2.1.2 LPM 驱动配置

1. **LPM 驱动是否使用**

对于不使用 LPM 驱动的平台，应确保任何时候 BT_WAKE 引脚电平是 ASSERT 状态，否则会造成蓝牙无法打开，或打开后又自行关闭的问题。

> 若不确定当前模组是否使用 LPM 驱动，请参考《Android_X_Wi-Fi_BT 开发指南》- 内核驱动配置章节，对于使用 LPM 驱动的模组，会在 menuconfig 有显示驱动使能开关。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. **LPM 驱动配置**

LPM 驱动典型的配置如下。其中，bt_wake 是输出引脚，连接 BT 的 AP_WAKE_BT 引脚，用作 AP 唤醒 BT 模组；bt_hostwake 是输入引脚，连接 BT 的 BT_WAKE_AP 引脚，配置为外部中断唤醒模式，用作 BT 模组唤醒 AP。注意，不同模组的 ASSERT LEVEL 可能不一样。如 XRADIO，必须配置为 1，1 模式；Broadcom ASSERT LEVEL 是可配置的，与上层对应即可；Realtek 大部分模组是 1，0 模式。具体模组的 ASSERT LEVEL 需要根据数据手册来确认。

```
&btlpm {
    compatible   = "allwinner,sunxi-btlpm";
    uart_index   = <0x1>;
    bt_wake      = <&r_pio PL 4 GPIO_ACTIVE_HIGH>;
    bt_hostwake  = <&r_pio PL 3 GPIO_ACTIVE_HIGH>;
    wakeup-source;
    status       = "okay";
};
```

图 4-25: LPM 配置

4.2.1.3 Firmware 错误

1. **Download Firmware 过程出错**

对于 Patch Ram 类型的芯片（Realtek/Broadcom），使用 HCI 命令来加载 Firmware。加载过程中有数据校验的过程，如果在该过程中报错，则一般是文件不匹配或文件损坏导致。

对于 Full Firmware 类型的芯片（XRADIO），使用私有协议来加载 Firmware。如该过程中出错，请直接联系全志获取支持。

2. **Firmware 文件错误**

典型的现象是，Stack 第一条 HCI 命令（HCI_RESET）就没有回应。请确保 Firmware 文件没有损坏，没有拷贝错误。如排除以上原因，请联系全志获取支持。

4.2.2 休眠唤醒类问题

4.2.2.1 无法休眠

该问题是指，打开蓝牙的情况下，机器无法进入休眠状态，但关闭蓝牙后，机器又可以正常进入休眠。排查点如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1. **LPM 引脚配置**

重点关注 ASSERT LEVEL 是否有配置错误的问题。

2. **位置服务**

Android 系统会周期性进行 LE 扫描以用于位置服务，该过程会阻止系统进入休眠。可以关闭位置服务内的蓝牙扫描加以确认问题。

4.2.2.2 无法唤醒

该问题是指，机器进入休眠状态后，无法通过所连接的蓝牙设备唤醒机器。排查点如下：

1. **LPM 驱动是否有打开**

唤醒功能依赖 LPM 驱动。如需该功能，请务必确保 LPM 驱动是编译并且正常加载的。

2. **power/io_regulator 休眠不能断电**

在机器休眠期间，BT 模组的 POWER 及 IO 供电不能断掉，否则机器唤醒功能不可用，并会引发唤醒后的蓝牙崩溃错误。

3. **host_wake 具有唤醒能力**

全志芯片只有 PL/PM 引脚具有休眠后中断唤醒能力，请确保硬件上 bt_hostwake 是连接到了 PL/PM GPIO 上，否则唤醒功能不可用。

4. **RTS/CTS 应按模组规范接电**

未按规范接电的问题表现为内核有被唤醒的动作，但很快又休眠下去，且该中断时刻上报的 event 在 hci 中并未有体现。如果蓝牙模组通过 uart 与 soc 通讯，这时就需要检查下该 uart 的 RTS、CTS **引脚接电是否符合规范**。

如对于模组 A 来说，无数据往来时，此两脚应为高电平状态。当 controller 要唤醒 host 时，会给出中断信号，并等待引脚指示后上报事件（即等待低电平信号）。但在实际电路中，此两脚接了下拉电阻。当有事件上报时，controller 给出中断信号，并因为 RTS/CTS 脚为低电平，误以为 host 已就绪，直接给 host 发送 event。然而此时 host 仍在处理中断，未准备好接收数据，就会导致上述异常。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.2.3 唤醒后挂掉

1. **休眠期间掉电**

   如果发生过掉电，唤醒后底层发生过掉电 reset，与上层状态不对应，引发 assert。

2. **唤醒后收到异常通信数据**

BT UART 连接到 Soc PG 口，其 RX 必须接上拉。原因如下：

AW SOC 休眠过程中，除了 PL/PM，其他 GPIO 控制器是掉电的，不受控。掉电后没有外部电路的情况下一般电平为 0。目前硬件设计上，Soc TX 与 BT Controller RX 直接相连，该引脚上没有上拉电阻。因此，在进入休眠后，GPIO 控制器掉电，Soc TX = 0。

BT 引起的唤醒时，是 BT 先唤醒，Soc 再唤醒，此时，BT Controller RX 上仍是低电平，会被 controller 认为是 uart 通信的 start bit。随后机器被唤醒，电平拉高，被 uart 识别为 stop bit。因此，会被 controller 收到 1byte 的异常数据。这个异常数据是不符合 hci 规范的，引起上层的 stack 报错，蓝牙重启掉。

```
40213 07-07 16:43:43.822  2077  2077 I          : Disabling non-boot CPUs ...
40214 07-07 16:43:43.823  2077  2077 I CPU1     : shutdown
40215 07-07 16:43:43.823  2077  2077 I psci     : CPU1 killed (polled 0 ms)
40216 07-07 16:43:43.825  2077  2077 I CPU2     : shutdown
40217 07-07 16:43:43.825  2077  2077 I psci     : CPU2 killed (polled 0 ms)
40218 07-07 16:43:43.828  2077  2077 I CPU3     : shutdown
40219 07-07 16:43:43.828  2077  2077 I psci     : CPU3 killed (polled 0 ms)
40220 07-07 16:43:43.830     0     0 W [    C0] PM: pm_system_irq_wakeup: 322 triggered bluetooth hostwake   bt host wake irq
40221 07-07 16:43:43.830  2077  2077 I          : Enabling non-boot CPUs ...
40222 07-07 16:43:43.830     0     0 I          : Detected VIPT I-cache on CPU1
40223 07-07 16:43:43.830     0     0 I CPU1     : Booted secondary processor 0x0000000001 [0x410fd034]   uart recv, controller -> libbt -> stack
40224 07-07 16:44:18.377  6532  7704 D bt_userial_vendor: ^^------, userial_recv_uart_rawdata(2342), libbt-to-stack, len: 17
40225 07-07 16:44:18.377  6532  7704 D bt_userial_vendor: 0000  04 04 0a 27 f2 72 ee 60  14 14 01 1a 01 04 10 01  |...'.r.`........|
40226 07-07 16:44:18.377  6532  7704 D bt_userial_vendor: 0001  00                                               |................|
40227 07-07 16:44:18.378  6532  7709 D android.hardware.bluetooth.hci_packetizer: ^^--------, recv data, type: 04
40228 07-07 16:44:18.379  6532  7709 D android.hardware.bluetooth-hci-h4: 0000  04 0a 27 f2 72 ee 60 14  14 01 1a 01   ok,connection request   |..'.r. .........|
40229 07-07 16:44:18.380  7651  7710 E bt_hci   : system/bt/main/bte_logmsg.cc:191 LogMsg: Ctlr H/w error event - code:0x0
40230 07-07 16:44:18.380  6532  7709 D android.hardware.bluetooth.hci_packetizer: ^^--------, recv data, type: 04
40231 07-07 16:44:18.380  7651  7710 E bt_btif : system/bt/main/bte_logmsg.cc:191 LogMsg: Received H/W Error.
40232 07-07 16:44:18.380  6532  7709 D android.hardware.bluetooth-hci-h4: 0000  10 01 00   hardware error   |................|
40233 07-07 16:44:18.382  6532  6532 D libbt_vendor: op for 7
40234 07-07 16:44:18.382  6532  6532 E bt_hwcfg_uart: set_wake_stat 1
```

图 4-26: 异常数据引发蓝牙挂掉

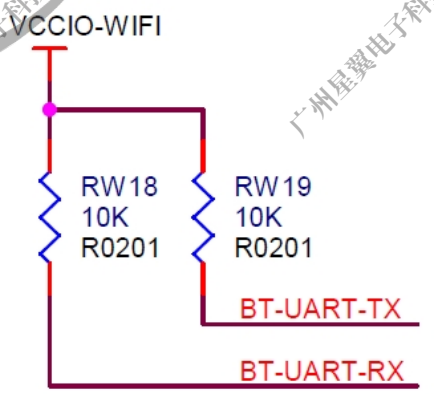

图 4-27: TX/RX 上拉电阻

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.3 设备兼容性

设备兼容性问题一般需要硬件及 Firmware 端的排查，需要抓取 HCI log 或空包进行专业的排查。此处仅介绍 log 的抓取，及可能的原因。

4.2.3.1 LOG 抓取

1. **HCI log 抓取方式**

首先打开 HCI 日志记录：Settings -> System -> Developer Options -> Enable Bluetooth HCI snoop log -> Enabled。然后重启下蓝牙，进行相关测试，问题复现后，从/data/misc/bluetooth/logs 取出 log。

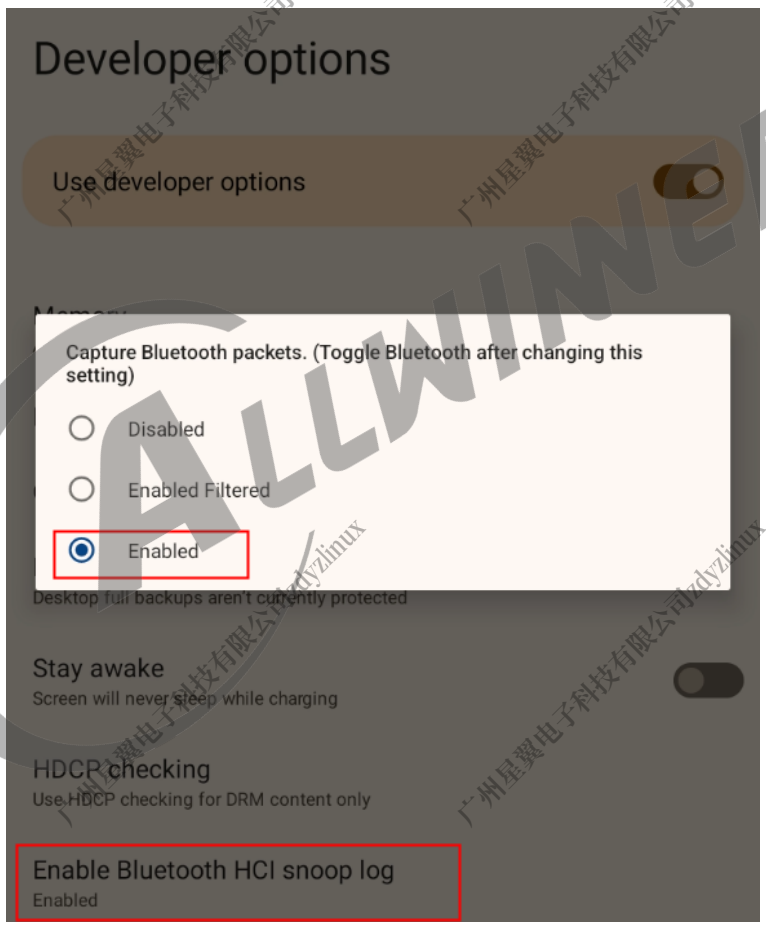

图 4-28: HCI log 设置

对于 user 固件，并无权限访问/data/misc/bluetooth/logs，可以提供 bugreport，里面包含该文件。

如果问题难以复现，且复现时恰好未开启 HCI LOG，请提供 bugreport 或者 dumpsys bluetooth_manager 的 log，里面包含系统自动记录的部分 HCI log。

2. **BPA600 空包抓取方式**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

空包一般需要比较昂贵的设备，如最常用的 BPA600，售价在 10W+。BPA600 抓包设置如下所示。注意，启动抓包后，需要手动开关下蓝牙，以抓取到必要的同步包。

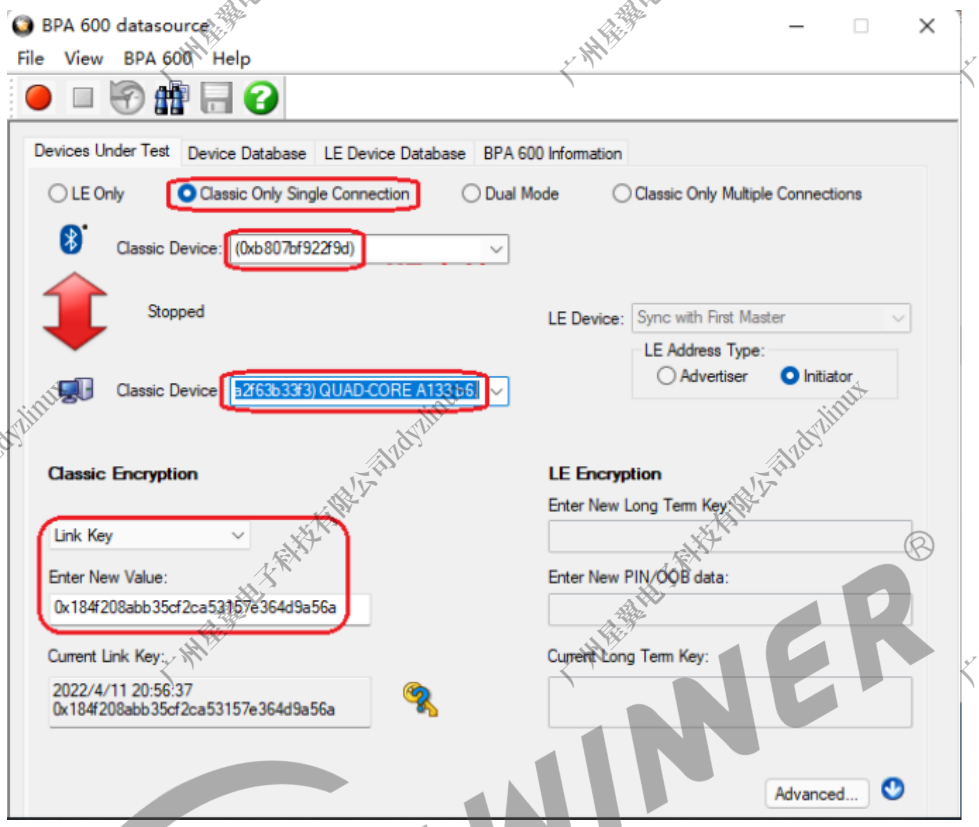

图 4-29: 抓空包设置

4.2.3.2 所有设备连接困难

1. **频偏**

通常，Wi-Fi/BT 公用射频电路。可通过 Wi-Fi 的频偏状况确认 BT 是否存在频偏问题。

2. **Firmware 是否正确**

确认 Firmware 的获取途径及正确性。

4.2.3.3 部分设备连接困难

请抓取 HCI log 或空包，并联系厂商获取支持。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。